

Analyse des processus - Exercice systèmes d'exploitation

Introduction

Cet exercice analyse 5 programmes en C utilisant l'appel système `fork()`. Pour chaque programme, on détermine :

- Le nombre total de processus créés
- L'arbre généalogique des processus
- Les valeurs des variables pour chaque processus

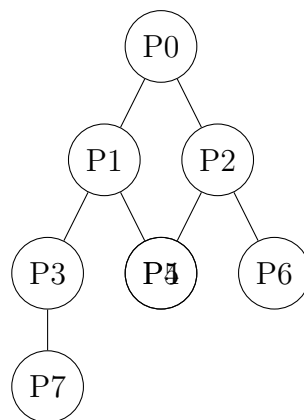
Code 1

```
int main() {  
    fork();  
    fork();  
    fork();  
}
```

Nombre de processus : 8

Trois appels successifs à `fork()` sans condition créent $2^3 = 8$ processus.

Arbre généalogique



Valeurs des variables

Aucune variable dans ce code.

Code 2

```
int main() {  
    int x, y, z;  
    x = fork();  
    y = fork();  
    z = fork();  
}
```

Nombre de processus : 8

Même structure que le Code 1 : 3 forks successifs $\rightarrow$ 8 processus.

Arbre généalogique

Même arbre que le Code 1.

Valeurs des variables

- Chaque processus a son propre triplet (x, y, z) :
- Dans le parent : `fork()` retourne le PID de l'enfant (`!0`)
 - Dans l'enfant : `fork()` retourne 0
 - Exemple (avec PID hypothétiques) :
 - P0 : x = PID1, y = PID2, z = PID3
 - P1 : x = 0, y = PID4, z = PID5
 - P2 : x = PID1, y = 0, z = PID6
 - P3 : x = 0, y = 0, z = PID7
 - etc.

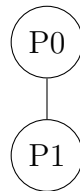
Code 3

```
int main() {  
    int x = 0;  
    int y = 0;  
    if (fork() > 0)  
        x = 2;  
    else  
        y = 4;  
}
```

Nombre de processus : 2

Un seul appel à `fork()` $\rightarrow$ 2 processus (parent + enfant).

Arbre généalogique



Valeurs des variables

- P0 (parent) : $x = 2$, $y = 0$
- P1 (enfant) : $x = 0$, $y = 4$

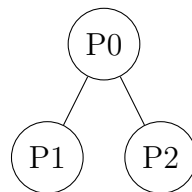
Code 4

```
int main() {  
    if (fork() > 0)  
        fork();  
}
```

Nombre de processus : 3

- Premier `fork()` : crée P1 (enfant)
- P0 (parent) entre dans le `if` et exécute le deuxième `fork()` : crée P2
- P1 ne rentre pas dans le `if` (car `fork()`=0)

Arbre généalogique



Valeurs des variables

Aucune variable dans ce code.

Code 5

```
int main() {  
    int cpt = 0;  
    while (cpt < 3) {  
        if (fork() > 0)  
            cpt++;  
        else  
            continue;  
    }  
}
```

```

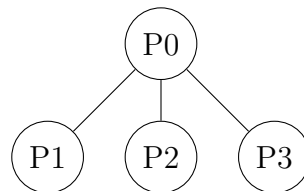
        cpt = 3;
    }
}

```

Nombre de processus : 4

- Itération 1 : P0 crée P1, P0.cpt = 1
- Itération 2 : P0 crée P2, P0.cpt = 2
- Itération 3 : P0 crée P3, P0.cpt = 3 → sortie de boucle
- P1, P2, P3 mettent cpt = 3 et sortent immédiatement

Arbre généalogique



Valeurs des variables

- P0 : cpt = 3 à la fin
- P1, P2, P3 : cpt = 3 dès leur création

Récapitulatif

Code	Nb processus	Arbre	Valeurs des variables
1	8	P0 → 8 processus	Aucune variable
2	8	P0 → 8 processus	x, y, z différents selon processus (0 ou PID)
3	2	P0 → P1	Parent : x=2,y=0; Enfant : x=0,y=4
4	3	P0 → P1 et P2	Aucune variable
5	4	P0 → P1, P2, P3	P0 : cpt=3; P1,P2,P3 : cpt=3

Conclusion

L'analyse montre que le comportement des programmes avec `fork()` dépend fortement :

1. Du nombre d'appels à `fork()`
2. Des conditions qui contrôlent l'exécution de `fork()`
3. De la logique des boucles et des incréments de variables

Le parent et l'enfant partagent les mêmes valeurs initiales de variables, mais chaque processus modifie ensuite ses propres copies indépendamment.